SOFTWARE ENGINEERING LAB

EXPERIMENT NO. 4

Project-Based Data Flow Diagram

kvstore

Distributed Key-Value Store with AI-Powered Observability

Aim

To prepare the Context Diagram, Level 1 subsystem data-flow diagram, and Level 2 internal package architecture for the selected software project, `kvstore`.

Name	Neha Nupur
Roll Number	20251651064
Programme	MCA, 3rd Semester
Subject Code	CS545
Subject	Software Engineering
Academic Session	2026

Prepared with reference to the supplied Lab 4 handout, SRS, and original `kvstore` DFD images.

Contents

1 Aim **2**

2 Objectives **2**

3 Project Details **2**

3.1 System scope 2

4 External Entities and Data Exchange **3**

5 Major Processes **3**

6 Data Store **3**

7 Context Diagram **4**

7.1 Context diagram interpretation 4

8 Level 1: Subsystem Data Flow **5**

8.1 Process interpretation 5

9 Level 2: Internal Package Architecture **6**

9.1 Package interpretation 6

10 Balancing Check **7**

11 Final Review Checklist **7**

12 Result **7**

1 Aim

To document the supplied Context Diagram, Level 1 subsystem data-flow diagram, and Level 2 internal package architecture for the `kvstore` project.

2 Objectives

- Represent the complete `kvstore` system and external exchanges in the Context Diagram.
- Explain the four major subsystems shown in the supplied Level 1 data-flow diagram.
- Describe the internal packages and their relationships in the supplied Level 2 architecture.
- Identify external entities, data flows, and persistent storage relevant to the project.
- Check that the diagrams are balanced and that process names and data flows are consistent across levels.

3 Project Details

Project title	`kvstore` – Distributed Key-Value Store with AI-Powered Observability
Purpose	A distributed, strongly consistent key-value store that replicates data using Raft consensus and provides an operational AI assistant to detect anomalies and suggest possible root causes to cluster operators.
Main external entities	Client Application; Cluster Operator / Administrator; External LLM Provider.
Major functions	API and Routing; Consensus (Raft); Storage (WAL, HLC, and MVCC); Observability (metrics and AI assistant).
Primary data store	D1: Persistent Storage, containing committed key-value entries, WAL records, and MVCC-versioned data.

3.1 System scope

The system accepts client key-value requests, routes them to the appropriate node or shard, uses consensus to replicate and commit proposals, stores committed data durably, and exposes operational metrics and insights. The AI assistant may send anomaly evidence to an external LLM provider and receive root-cause hypotheses. The LLM provider is an external dependency; it does not directly commit data or change consensus state.

4 External Entities and Data Exchange

External entity	Data sent to `kvstore`	Data received from `kvstore`
Client Application	Key-value requests: Put, Get, Delete, and supported transaction requests; request metadata.	Transaction outcomes, values, acknowledgements, and error responses.
Cluster Operator / Administrator	Monitoring queries and operational questions, when supported by the dashboard/API.	Live dashboard, metrics, alerts, insights, and operational explanations.
External LLM Provider	Root-cause hypotheses or analysis returned in response to submitted evidence.	Anomaly evidence and bounded diagnostic context supplied by the observability component.

Boundary note: The external LLM is used for diagnostic assistance only. Consensus decisions, transaction commits, and persistent writes remain controlled by the `kvstore` system.

5 Major Processes

Process	Name	Responsibility
1.0	**API & Routing**	Handles HTTP/gRPC transport, validates and routes client requests, follows leader redirects, and resolves shard destinations. Internal packages include `httpapi/transport`, `client`, and `shard`.
2.0	**Consensus**	Runs Raft-based leader election and log replication; coordinates proposal commitment and uses the write-ahead log for durability.
3.0	**Storage**	Applies committed entries, persists key-value state, manages hybrid logical clock (HLC) timestamps and multi-version concurrency control (MVCC).
4.0	**Observability**	Collects metrics and performance signals, surfaces alerts and insights, and integrates an AI-assisted anomaly/root-cause workflow.

6 Data Store

ID	Name	Data stored
D1	**Persistent Storage**	Committed key-value entries, write-ahead log (WAL) records, and MVCC-versioned data. WAL records are appended durably; the storage layer applies committed entries to the state representation.

DFD convention: External entities interact with processes, not directly with data stores. The diagrams therefore route requests through API & Routing and route committed state through Consensus / Storage.

7 Context Diagram

The Context Diagram treats `kvstore` as one system and shows the external entities and the major data exchanged with the system. The image below is reproduced directly from the supplied project diagram; its shapes, labels, and connectors have not been redrawn.

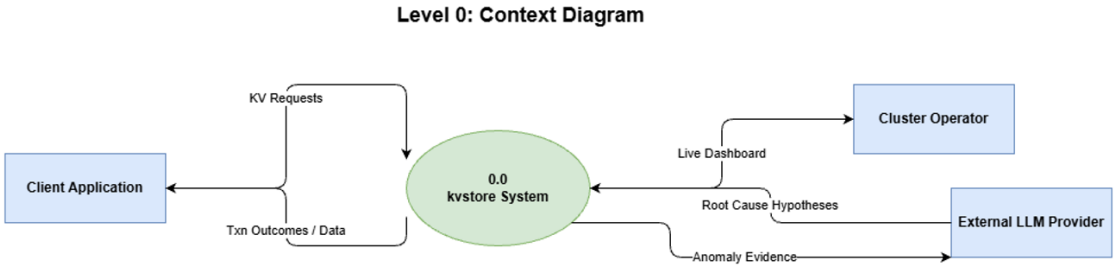


Figure 1: Original Context Diagram supplied for `kvstore`

7.1 Context diagram interpretation

- The Client Application sends key-value requests and receives transaction outcomes or data.
- The Cluster Operator receives the live dashboard information.
- The External LLM Provider receives anomaly evidence and returns root-cause hypotheses.

8 Level 1: Subsystem Data Flow

This supplied diagram shows the primary subsystem flow through API & Routing, Consensus, Storage, and Observability. It is inserted as the original image rather than reconstructed in TikZ, preserving the provided labels and arrow directions.

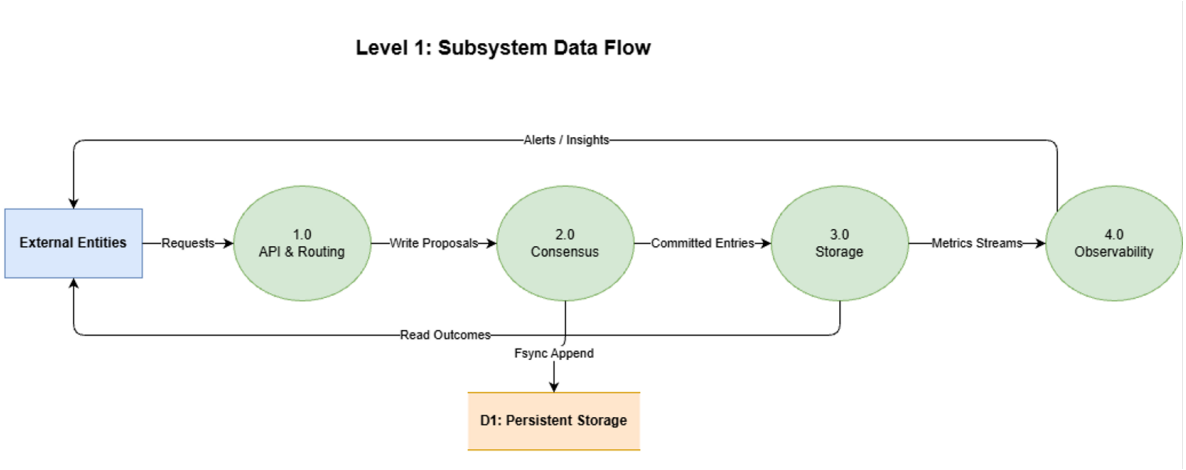


Figure 2: Original Level 1 subsystem data-flow diagram supplied for `kvstore`

8.1 Process interpretation

1. **1.0 API & Routing:** accepts requests from external entities and routes them to the relevant subsystem.
2. **2.0 Consensus:** coordinates proposals and consensus decisions.
3. **3.0 Storage:** handles committed entries and persistent state.
4. **4.0 Observability:** consumes metric streams and returns alerts or insights to external entities.

The diagram also shows the persistent store D1 and the *Fsync Append* flow. The data-flow labels are retained exactly as they appear in the supplied image.

9 Level 2: Internal Package Architecture

The following is the original package-architecture image supplied with the project. It is included without redrawing or altering its internal nodes and connectors.

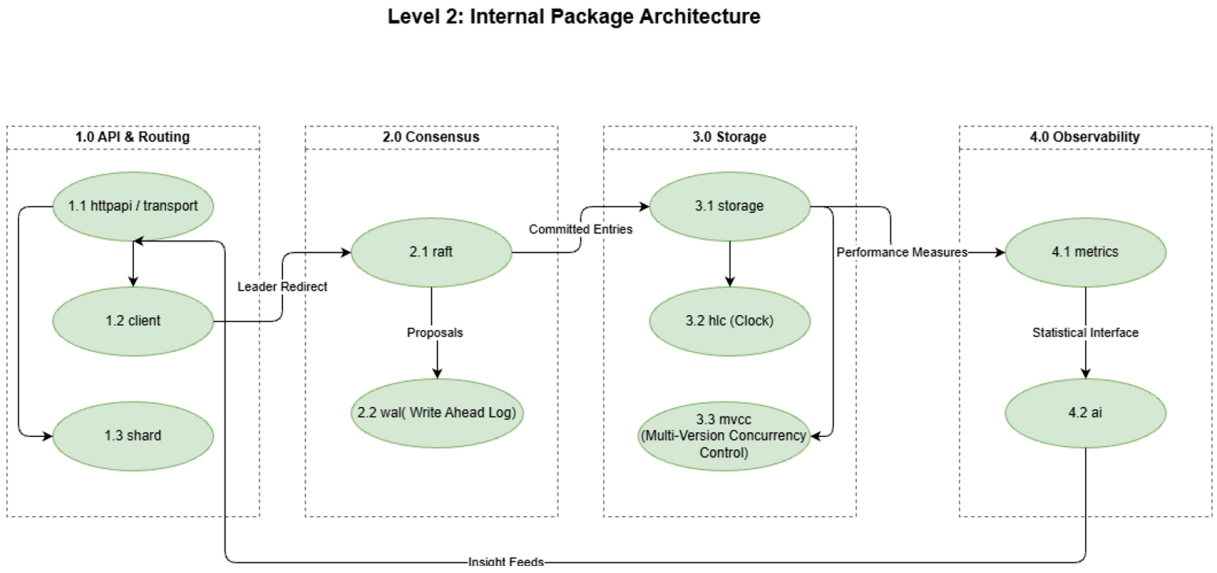


Figure 3: Original Level 2 internal package architecture supplied for `kvstore`

9.1 Package interpretation

- **API & Routing:** contains the HTTP API/transport, client, and shard components.
- **Consensus:** contains Raft and the write-ahead log (WAL).
- **Storage:** contains storage, HLC clock, and MVCC components.
- **Observability:** contains metrics and the AI component, connected by the statistical interface.

10 Balancing Check

Check	Status	Explanation
Context Diagram and Level 0 are consistent.	Yes	Client requests and outcomes remain represented; operator monitoring and LLM diagnostic exchanges are preserved when internal processes are shown.
Selected Level 0 process and Level 1 are consistent.	Yes	Level 1 expands Consensus into Raft and WAL while preserving proposals as input and committed entries as output.
No unrelated data appears in Level 1.	Yes	The sub-processes refine consensus and durability behaviour only.
External entities connect through processes.	Yes	No external entity is connected directly to D1.
Every important flow is labelled.	Yes	Labels describe data such as requests, proposals, committed entries, metrics, and hypotheses rather than generic actions.

11 Final Review Checklist

✓ The work is based on the selected project, `kvstore`.

✓ Process names match the system's major functions.

✓ Arrows represent labelled data flows.

✓ External entities do not connect directly to the data store.

✓ The supplied Context Diagram, Level 1 subsystem DFD, and Level 2 package architecture are included.

✓ The relationship between the supplied diagram levels is described and the balancing check is documented.

12 Result

The `kvstore` project has been documented using the supplied Context Diagram, Level 1 subsystem data-flow diagram, and Level 2 internal package architecture. The diagrams identify the Client Application, Cluster Operator, and External LLM Provider; show the API & Routing, Consensus, Storage, and Observability subsystems; and provide internal package detail for Raft, WAL, HLC, MVCC, metrics, and AI. The diagrams are reproduced from the provided images so their original labels and connectors are preserved.